Drift-Aware Adaptive Scheduling in 5G Open RAN using Unsupervised Anomaly Detection

Akshat Dodwad[1], Naveen Huggi[1], Abhishek M.[1], Riya A. Magadum[1], Narayan D. G.[1*]

[1*]School of Computer Science and Engineering, KLE Technological University, Hubballi, 580031, Karnataka, India.

*Corresponding author(s). E-mail(s): narayan.dg@kletech.ac.in;
Contributing authors: akshat.dodwad@gmail.com; naveen.huggi@gmail.com; abhishek.m@gmail.com; riya.magadum@gmail.com;

Abstract

The dynamic nature of 5G networks, characterized by diverse services and fluctuating channel conditions, often renders static scheduling algorithms ineffective due to "concept drift." This drift inevitably leads to a degraded Quality of Service (QoS), reduced throughput, and unfair resource distribution. While existing machine learning solutions attempt to optimize scheduling, they predominantly rely on supervised learning models that require vast amounts of labeled data—a critical limitation since real-world network metrics are mostly unlabelled. Furthermore, few studies focus on autonomously improving network performance precisely when concept drift occurs. To bridge this gap, this paper proposes a novel "Drift-Aware Adaptive Scheduling" framework within the Open Radio Access Network (O-RAN) architecture. Leveraging the Near-Real-Time RAN Intelligent Controller (Near-RT RIC), we introduce a Machine Learning (ML) xApp that utilizes an unsupervised Isolation Forest algorithm to monitor Key Performance Indicators (KPIs) such as throughput and Physical Resource Block (PRB) utilization. Our approach detects statistical anomalies in real-time using sliding windows of unlabelled telemetry data. Upon identifying sudden or gradual drift, the system autonomously tunes MAC-layer scheduling parameters via the E2 interface, closing the control loop. Implemented on a gNodeB-based O-RAN testbed with OpenAirInterface and FlexRIC, our experimental results demonstrate that the proposed drift-aware scheduler achieves a 15% improvement in average user throughput and maintains an excellent Jain's Fairness Index of

0.9994 under congestion. This study proves the efficacy of unsupervised, data-driven optimization for maintaining robust 5G network performance in highly volatile environments.

Keywords: 5G, Open RAN, Resource Scheduling, Concept Drift, Unsupervised Learning, Isolation Forest, FlexRIC, Near-RT RIC

1 Introduction

The deployment of 5G networks has introduced a paradigm shift in telecommunications, fundamentally altering how mobile networks are designed, deployed, and managed. Characterized by the need to support high-performance, diverse applications, 5G architectures must seamlessly cater to three primary use cases: Ultra-Reliable Low Latency Communications (URLLC) for mission-critical industrial automation, enhanced Mobile Broadband (eMBB) for high-definition video streaming, and massive Machine Type Communications (mMTC) for Internet of Things (IoT) deployments. To accommodate these diverse and often conflicting Service Level Agreements (SLAs), network orchestration must be exceptionally dynamic. However, despite the advanced physical layer capabilities of 5G New Radio (NR), gNodeB-based resource management at the Radio Access Network (RAN) layer frequently relies on legacy, static scheduling algorithms, such as Round Robin (RR), Maximum Rate (MaxRate), or Proportional Fair (PF).

These traditional algorithms are inherently rigid; they perform adequately under stable, predictable traffic conditions but falter significantly in real-world scenarios where user behavior, mobility patterns, and radio channel conditions rapidly fluctuate. For instance, while the Proportional Fair scheduler attempts to balance overall cell throughput against individual user fairness, it operates on static mathematical weights that cannot adapt to sudden network congestion or unpredictable shifts in spectral efficiency. Consequently, this rigidity leads to suboptimal resource distribution, resulting in severe performance bottlenecks, reduced edge-user throughput, increased queuing latency, and ultimately, an unacceptable degradation of the user experience during peak load conditions.

To address these critical limitations, modern 5G architectures, particularly the Open Radio Access Network (O-RAN) paradigm, are increasingly integrating Artificial Intelligence (AI) and Machine Learning (ML) models to enable intelligent, data-driven scheduling. The O-RAN architecture facilitates this by decoupling the intelligent control plane from the underlying radio hardware, allowing network operators to deploy complex algorithms directly within the RAN via localized controllers. However, the practical application of ML in highly dynamic wireless environments introduces a profound challenge known as *concept drift*. Concept drift occurs when the underlying statistical properties of network traffic and channel states change over time due to external factors—such as a sudden influx of users, changes in application usage trends, or physical topological shifts—rendering previously trained predictive models outdated and ineffective.

If an ML-based scheduler continues to make radio resource allocation decisions based on obsolete historical data patterns, the network's Quality of Service (QoS) will inevitably and catastrophically degrade. A model trained during a period of low traffic will misallocate Physical Resource Blocks (PRBs) when sudden congestion occurs, leading to high packet drop rates and starvation for edge users. Therefore, there is an imperative need for intelligent scheduling systems capable of autonomous, real-time adaptation—systems that can continuously monitor their own live performance metrics, accurately detect statistical shifts in the data distribution, and dynamically adjust their scheduling strategies at the exact moment concept drift is detected.

To address these critical gaps, this paper aims to design and develop a novel, unsupervised drift-aware scheduling algorithm for 5G O-RAN. The primary objective is to create a closed-loop system that autonomously monitors unlabelled Key Performance Indicators (KPIs) in real-time, accurately detects statistical concept drift without relying on heavy offline retraining, and adaptively optimizes radio resource allocation to balance system throughput, user fairness, and latency. By decoupling the intelligent control logic (via an xApp) from the physical gNodeB hardware, we seek to demonstrate that continuous, unsupervised anomaly detection can effectively maintain optimal 5G network performance in the face of unpredictable environmental changes.

The explicit contributions of this work are summarized as follows:

- **Unsupervised Drift Detection Framework:** We propose a lightweight, unsupervised Isolation Forest algorithm combined with Exponential Moving Average (EMA) smoothing to detect network concept drift in real-time using unlabelled gNodeB telemetry.
- **Dynamic Closed-Loop MAC Scheduling:** We design a policy enforcement engine that translates detected drift severity into actionable, real-time E2 control messages, dynamically scaling Proportional Fair weights and restricting PRB limits to prevent user starvation.
- **High-Fidelity Testbed Validation:** We implement and evaluate the proposed drift-aware adaptive scheduler on a live 5G O-RAN testbed utilizing OpenAir-Interface (OAI) and FlexRIC, proving its superiority over static schedulers and conventional supervised drift-handling models in terms of latency, throughput, and fairness.

2 Background Study

This section provides the foundational concepts required to understand the proposed drift-aware scheduling framework. First, we outline the architectural evolution of 5G and the paradigm shift introduced by the Open Radio Access Network (O-RAN). Subsequently, we present a comprehensive survey of the existing literature concerning resource scheduling, machine learning applications in telecommunications, and the critical challenge of concept drift, highlighting the gaps that this paper aims to address.

5G Architecture and O-RAN. The evolution from traditional, monolithic 4G/LTE networks to 5G has necessitated a fundamental redesign of the Radio Access Network. The Open Radio Access Network (O-RAN) architecture introduces a paradigm shift by disaggregating the traditional base station (eNodeB/gNodeB)

into distinct, interoperable, and software-defined components: the Radio Unit (O-RU), Distributed Unit (O-DU), and Centralized Unit (O-CU). This disaggregation is defined by strict functional splits (e.g., Option 7-2x between the RU and DU), allowing for multi-vendor interoperability and the virtualization of network functions on commercial off-the-shelf (COTS) hardware.

Crucially, this structural disaggregation is coupled with the integration of RAN Intelligent Controllers (RICs), which enable intelligent, data-driven management of network resources. The O-RAN architecture defines two primary intelligence layers: the Non-Real-Time (Non-RT) RIC, which handles policy management and model training at latency scales greater than 1 second, and the Near-Real-Time (Near-RT) RIC, which operates at latency scales between 10 milliseconds and 1 second. As illustrated in Figure 1, the Near-RT RIC acts as the localized brain of the RAN ecosystem. It allows for the deployment of third-party Machine Learning (ML) models as "xApps," which can directly monitor and control the underlying O-DU and O-CU instances. This closed control loop is facilitated by the standardized E2 interface, which supports high-frequency telemetry subscription (E2SM-KPM) and direct policy enforcement (E2SM-RC), empowering our proposed unsupervised drift-aware scheduler to dynamically manipulate MAC-layer resource allocations in response to fluctuating network conditions.

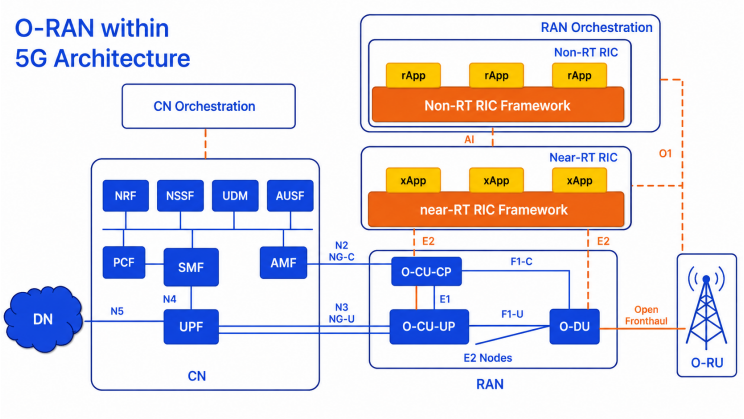

Fig. 1 5G Open RAN Architecture and Intelligent Control Loop

Related Work. The literature surrounding 5G Radio Access Network (RAN) resource scheduling and optimization is extensive, reflecting the critical need to meet diverse and stringent Service Level Agreements (SLAs). With the advent of the O-RAN architecture, research has increasingly shifted from static, monolithic scheduling algorithms toward intelligent, data-driven approaches. In this section, we provide a comprehensive review of the evolution of these scheduling paradigms. We categorize the existing literature into six primary domains: traditional and heuristic scheduling, supervised learning for traffic prediction, deep reinforcement learning and federated approaches, fairness and Quality of Service (QoS) management, concept drift and

dynamic adaptation, and energy-efficient multi-objective optimization. This structured analysis highlights the progression of RAN intelligence and explicitly identifies the critical gaps that motivate our unsupervised drift-aware framework.

Traditional and Heuristic Scheduling Approaches. Historically, base station scheduling has relied heavily on well-established heuristic algorithms designed to balance throughput and fairness under predictable channel conditions. The Proportional Fair (PF) scheduler, Maximum Rate (MaxRate), and Round Robin (RR) algorithms have served as the foundational pillars of 4G LTE and early 5G deployments. Recent studies have attempted to push the boundaries of these static frameworks. For instance, Al-Zubaidy et al. [1] proposed an advanced beam division multiple access scheduling algorithm tailored for highly congested traffic scenarios. Their approach demonstrated capacity improvements by leveraging spatial multiplexing; however, it inherently ignores critical interference and time-varying noise effects, which severely limits its real-world applicability in unpredictable, highly mobile urban environments.

Similarly, Orumwense et al. [7] developed an optimal heuristic scheduling technique specifically aimed at supporting smart grid communications over 5G networks. While effective for the specific deterministic traffic patterns of smart grids, these traditional algorithms suffer from profound rigidity. They depend on fixed mathematical weights and static utility constraints that cannot dynamically respond to sudden network congestion, user mobility spikes, or unforeseen environmental changes. When subjected to volatile conditions, these schedulers frequently cause resource starvation for cell-edge users, leading to degraded fairness and unacceptable latency. The fundamental limitation of heuristics is their lack of contextual awareness; they react to instantaneous channel quality indicators without anticipating broader network trends or shifting traffic distributions.

Supervised Learning and Traffic Prediction in RAN. To overcome the limitations of static heuristics, researchers have extensively explored supervised Machine Learning (ML) models to proactively predict traffic loads and optimize resource slicing. Supervised learning models, particularly Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks, have shown significant promise in forecasting temporal traffic patterns. Samson et al. [3] introduced a hybrid LSTM and game theory synergy framework designed to optimize 5G network slices. By predicting future load, their system attempts to preemptively allocate resources to avoid congestion. In a similar vein, Chen et al. [4] proposed a channel-aware 5G RAN slicing framework that integrates customizable predictive schedulers, proving that proactive allocation can significantly improve spectral efficiency compared to reactive methods.

Further extending this paradigm, Lotfi et al. [6] explored LSTM-based traffic prediction combined with slice management, aiming to provide strict QoS guarantees for diverse use cases. While these supervised predictive frameworks demonstrate high accuracy in simulation, they suffer from a fundamental flaw when deployed in live O-RAN environments: they require massive volumes of perfectly labeled, historically relevant training data. In real-world networks, unlabelled telemetry is the norm, and supervised models struggle to generalize to unseen topological anomalies. Moreover, these studies predominantly focus on macroscopic traffic prediction over intervals of

minutes or hours, rather than addressing the micro-second scale closed-loop mobility or the sudden onset of concept drift at the MAC layer.

Deep Reinforcement Learning and Federated Approaches. The shift toward autonomous, intent-based network management has heavily popularized Deep Reinforcement Learning (DRL). By interacting directly with the network environment, DRL agents learn optimal scheduling policies through trial and error, bypassing the need for labeled datasets. Tsampazi et al. [21] introduced Pandora, an automated design framework for evaluating DRL agents as xApps within O-RAN, demonstrating that centralized reinforcement can outperform static PF schedulers in aggregate throughput. Furthermore, modern automated scheduling frameworks like Allstar [8] facilitate Large Language Model (LLM)-driven intent-based scheduler generation and testing, streamlining the deployment of intelligent agents.

However, centralizing the intelligence at the Near-RT RIC often introduces latency bottlenecks. To mitigate this, decentralized and federated reinforcement approaches have been proposed. Arslan et al. [10] proposed a dynamic MAC scheduling framework in O-RAN using federated deep reinforcement learning, where local agents learn on individual gNodeBs and periodically sync with a global model. Zhang et al. [20] and Mudi et al. [19] explored federated resource allocation frameworks for O-RAN slicing and metaverse applications, respectively. Meta-RL frameworks by Lotfi et al. [11] also attempt to improve generalization across varied network states by training a meta-policy capable of rapid adaptation.

Despite their theoretical sophistication, DRL and federated frameworks face massive communication and computational bottlenecks in practice. These models require constant, iterative synchronization of neural network weights across the E2 and O1 interfaces. During periods of peak traffic, this heavy signaling overhead can saturate the control plane, actively increasing latency precisely when the network is struggling the most. As highlighted by Tsampazi et al. [14] in their comparative analysis, the training and execution overhead of DRL xApps often renders them too slow to react to instantaneous channel degradation, making them unsuitable for ultra-reliable low-latency communications (URLLC).

Fairness and Quality of Service (QoS) Management. Ensuring fairness among users while maximizing overall system throughput is a classic optimization problem in wireless networks. Traditional approaches, such as the Proportional Fair algorithm, attempt to strike a balance but often fail during periods of extreme congestion, leading to the starvation of cell-edge users. Recent literature has sought to integrate intelligent fairness mechanisms into RAN scheduling. For instance, QoS-aware slicing mechanisms have been designed to isolate resources for mission-critical applications, but strict isolation often leads to spectral underutilization.

In dynamic environments, maintaining a high Jain's Fairness Index (JFI) requires continuous adaptation of resource allocation bounds. Real-time adaptation frameworks like 5G-TPS [9] provide a two-phase scheduling approach to handle changing conditions by continuously reassessing user utility functions. However, they rely on strictly structured traffic assumptions and mathematical utility functions that may not hold true under concept drift. The challenge remains to develop a scheduling framework

that guarantees fairness without introducing the computational overhead of iterative utility optimization or the instability of heuristic tuning.

Concept Drift and Dynamic Adaptation. Transitioning ML models from controlled simulation environments to practical O-RAN deployments exposes them to *concept drift*—the phenomenon where the statistical properties of the target variable change over time, degrading model accuracy. For instance, a model trained during morning traffic patterns will perform poorly during a sudden evening sports event. While Polese et al. [5] successfully demonstrated the foundational feasibility of deploying gNodeB-level scheduling xApps in a real O-RAN testbed (Colo-RAN), their seminal work primarily focused on basic system integration and E2 signaling validation, leaving long-term model decay unaddressed.

Efforts to tackle concept drift explicitly are still in their infancy in the telecommunications domain. The framework proposed by Gudepu et al. [2] is a notable exception; they developed a drift handling framework for O-RAN that relies on tracking supervised learning error metrics like Root Mean Square Error (RMSE). When the prediction error exceeds a hand-tuned threshold, the system triggers a full offline model retraining cycle or switches between pre-certified models. While effective in theory, this approach introduces substantial computational overhead and prolonged delays (often tens of seconds). During the retraining window, the network is left operating suboptimally, which can be catastrophic.

Online learning frameworks such as SORA [16] attempt to adapt on the fly but are highly prone to instability and "catastrophic forgetting" in volatile traffic conditions, frequently failing to maintain fairness when sudden congestion strikes. Furthermore, the push for explainable AI in O-RAN, as discussed by Fiandrino et al. (Explora) [12], emphasizes the need for models whose adaptation mechanisms are transparent and predictable—a requirement that complex DRL agents and opaque online learning models often fail to meet.

Energy Efficiency and Multi-Objective Optimization. Recently, a new dimension of complexity has been added to RAN scheduling: energy efficiency. As 5G deployments densify, the power consumption of base stations has become a critical operational concern. Liang et al. [13] investigated enhancing energy efficiency in O-RAN through intelligent xApp deployment, focusing on turning off redundant radio units during off-peak hours. Lozano et al. (Kairos) [17] proposed energy-efficient radio unit control via advanced sleep modes at the micro-second level. Additionally, Metere et al. [18] explored formal verification methods to achieve a balance between energy efficiency and service availability in future 6G O-RAN architectures. Transfer learning techniques have also been proposed by Sherif et al. [15] to accelerate the deployment of efficient O-RAN slices across different geographical regions. While these works contribute significantly to the green networking initiative, optimizing for energy often directly conflicts with maintaining peak throughput and fairness during sudden traffic spikes, further highlighting the need for highly adaptive, multi-objective scheduling frameworks capable of real-time trade-offs.

Summary of Research Gaps. Despite these collective advancements across heuristics, supervised prediction, DRL, fairness optimization, and energy efficiency, a critical analysis reveals two profound research gaps that severely limit the practical

deployment of intelligent 5G schedulers. First, while numerous papers discuss detecting network anomalies or integrating complex agents, **none of the existing literature focuses on actively and autonomously improving the network's scheduling policy precisely at the moment concept drift occurs using lightweight techniques**. Most existing solutions simply trigger an offline retraining pipeline, leaving the network vulnerable and degraded during the entire retraining window. Second, there is a distinct **lack of unsupervised learning approaches** for real-time drift adaptation. Existing ML schedulers are predominantly supervised, requiring perfectly labeled datasets that do not exist in live deployments. Relying on supervised learning creates a massive bottleneck where the gNodeB cannot adapt to novel, unseen traffic patterns or unpredictable user mobility. This paper directly addresses these critical gaps by introducing a fully unsupervised, lightweight scheduling architecture that adapts to drift instantly without the need for labeled data or costly neural network retraining.

3 Mathematical Formulation

The network state at any time t can be mathematically represented as a feature vector $x_t \in \mathbb{R}^d$ consisting of d Key Performance Indicators (KPIs):

$$x_t = [k_t^1, k_t^2, \ldots, k_t^d] \tag{1}$$

Specifically, for our drift-aware scheduling system, the feature vector is defined as:

$$x_t = [\text{DL Throughput}, \text{UL Throughput}, \text{DL PRB}, \text{UL PRB}] \tag{2}$$

Because real-world network telemetry lacks explicit labels for anomalous traffic states or congestion events, supervised learning models are impractical for live deployment. Therefore, our system employs an unsupervised Isolation Forest algorithm [22]. The network state at any time t is represented by a feature vector x_t comprising the filtered KPIs. The Isolation Forest isolates anomalous observations by randomly selecting features and corresponding split values. The anomaly score $s(x_t) \in [0, 1]$ for a given feature vector x_t is defined mathematically as:

$$s(x_t) = 2^{-\frac{\mathbb{E}(h(x_t))}{c(n)}} \tag{3}$$

Where:

- $h(x_t)$ is the path length, representing the number of edges x_t traverses from the root to an external node in an isolation tree.
- $\mathbb{E}(h(x_t))$ is the average path length of x_t across the ensemble of isolation trees.
- $c(n)$ is the average path length of an unsuccessful search in a Binary Search Tree (BST) built with n nodes (the training dataset size), which serves as a normalization factor:

$$c(n) = 2H(n-1) - \frac{2(n-1)}{n} \tag{4}$$

Here, the harmonic number $H(i)$ is estimated as $H(i) \approx \ln(i) + \gamma$, where $\gamma \approx 0.5772156649$ is Euler's constant.

The anomaly score $s(x_t) \in [0, 1]$ is interpreted as follows:

- $s(x_t) \to 1$: Highly anomalous behavior (abnormal state/congestion).
- $s(x_t) \to 0.5$: Baseline behavior, with no distinct anomaly.
- $s(x_t) \to 0$: Completely normal, densely clustered network behavior.

To mitigate transient noise and prevent erratic scheduling changes, the instantaneous anomaly scores are smoothed using an Exponential Moving Average (EMA) to compute a definitive Drift Score, D_t:

$$D_t = \lambda \cdot s(x_t) + (1 - \lambda) \cdot D_{t-1} \tag{5}$$

Where $\lambda \in (0, 1)$ is the smoothing factor controlling the response sensitivity of the controller.

Once the drift type is classified, the system mathematically optimizes an objective function designed to minimize drift-induced abnormal behavior while maximizing system throughput, strictly bounded by the total available PRBs. The optimization problem across evaluation intervals is formulated as:

$$\min(J) = \frac{1}{T} \sum_{t=1}^{T} (\alpha \cdot D_t - \beta \cdot \text{Throughput}_t) \tag{6}$$

Where:

- T is the total number of evaluation windows.
- D_t is the EMA-smoothed Drift Score at time t.
- Throughput_t is the measured downlink data transmission rate at time t.
- $\alpha, \beta \geq 0$ are dynamic weighting factors that control the trade-off between stability and throughput optimization.

To guarantee Quality of Service (QoS) and respect physical resource constraints, the system operates under the following mathematical constraints:

1. **Minimum Throughput Constraint**: Each active User Equipment (UE) $u \in \mathcal{U}$ must receive a minimum guaranteed throughput $R_{\min}$ to maintain service continuity:
$$\text{Throughput}_u \geq R_{\min}, \quad \forall u \in \mathcal{U} \tag{7}$$
(In this testbed, the minimum throughput is considered best-effort ($R_{\min} = 0$ Mbps) with a nominal reference target of 10 Mbps).
2. **Physical Resource Block (PRB) Constraint**: The total number of allocated resource blocks must not exceed the available channel resources of the gNodeB:
$$\sum_{u \in \mathcal{U}} \text{PRB}_u \leq N_{\text{PRB}}^{\text{total}} \tag{8}$$

(For a 10 MHz channel bandwidth with 15 kHz subcarrier spacing, the total available PRBs is $N_{\text{PRB}}^{\text{total}} = 52$).

4 Proposed work

To address the limitations of static heuristics and computationally heavy supervised learning models, we propose a novel Drift-Aware Adaptive Scheduling framework. This section details the complete methodology, starting with the overall system model within the O-RAN architecture. We then describe the unsupervised prediction model used for real-time drift detection and conclude with the closed-loop policy enforcement mechanism that dynamically adjusts scheduling parameters.

4.1 System Model

The proposed Drift-Aware Adaptive Scheduling framework is designed within the O-RAN architecture to continuously balance network stability and performance. Traditional monolithic RANs tightly couple the control plane with the radio hardware. In contrast, our architecture leverages the Near-Real-Time RAN Intelligent Controller (Near-RT RIC) to host a closed-loop intelligence mechanism. This mechanism consists of three primary components: a KPI Monitor xApp, a Drift Detection Engine, and a Scheduling Enforcement xApp. By decoupling the intelligent control logic from the physical gNodeB, the system achieves real-time responsiveness without bottlenecking the base station's core operations.

4.2 Prediction model

The foundation of the drift-aware system is the continuous stream of unlabelled telemetry data. The gNodeB exposes its internal state metrics via the standardized O-RAN E2 interface. The KPI Monitor xApp acts as the data ingress pipeline, receiving periodic telemetry samples that include Downlink/Uplink Throughput, Physical Resource Block (PRB) Utilization, and Reference Signal Received Power (RSRP). Upon reception, the raw data undergoes strict outlier filtering—physically invalid negative measurements are discarded, and values exceeding theoretical maximums are clamped. Only sanitized, temporally consistent samples are passed to the intelligent core.

This continuous KPI monitoring and pre-processing pipeline is described in Algorithm 1.

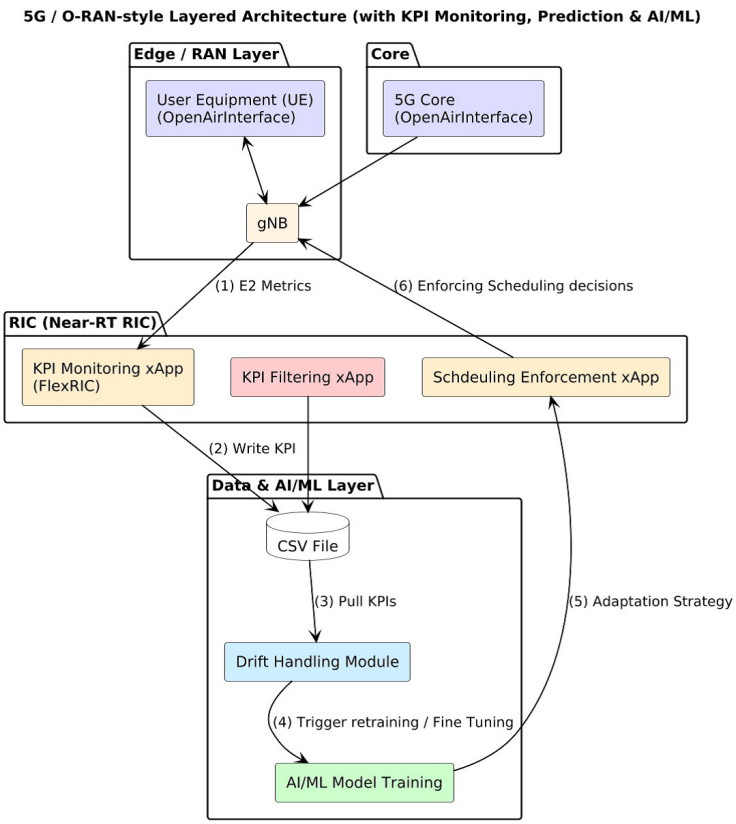

Fig. 2 Methodology Architecture and Closed-Loop Workflow of the Proposed Drift-Aware Scheduling System in 5G Open RAN

Algorithm 1 Real-Time Telemetry Subscription and Quality Filtering

Require: Raw Key Performance Indicator (KPI) metrics from the gNodeB

Ensure: Sanitized and pre-processed telemetry stream for downstream anomaly detection

1: **Initialization:** Establish a temporal connection to the Near-RT RIC and open a dedicated E2 interface session bridge to the gNodeB.

2: **Service Model Subscription:** Register the telemetry monitor service, subscribing to the standardized E2SM-KPM service model.

3: **Periodic Acquisition:** Trigger a background collection loop running at a synchronized polling interval of $\tau = 1$ second to acquire gNodeB telemetry.

4: **Quality Filtering and Clamping:** For each received metric vector:

5: Validate values against a predefined maximum threshold $(1,000,000)$. Any outlier exceeding this boundary is clamped to zero to prevent numeric instability.

6: Filter out negative noise and clamp negative measurements to zero.

7: **Database Logging:** Check the gNodeB reporting offset. If the offset is zero, append the sanitized KPI sample to a CSV database record.

8: **Downstream Propagation:** Forward the filtered telemetry sample to Prometheus and Grafana databases for real-time visualization and alert triggering.

Concept drift is classified by analyzing both the magnitude of D_t and its rate of change $|D_t - D_{t-1}|$ against a drift threshold θ and a rate parameter ϵ:

1. **Stable (No Drift)**: Normal network behavior where $D_t \leq \theta$.
2. **Gradual Drift**: Incremental degradation or slowly shifting traffic patterns over time, indicating a gradual change in user behavior where $D_t > \theta$ and $|D_t - D_{t-1}| < \epsilon$.
3. **Sudden Drift**: Abrupt network topology changes, such as a sudden influx or disconnection of UEs where $D_t > \theta$ and $|D_t - D_{t-1}| \geq \epsilon$.

This hybrid drift detection and classification logic is executed according to Algorithm 2.

Algorithm 2 Unsupervised Drift Detection and Severity Classification

Require: Unlabelled network metrics stream
Ensure: Drift severity classification in real-time

Feature Extraction: Extract the multi-dimensional KPI vector x_t containing downlink/uplink throughput and physical resource blocks from the sanitized telemetry sample.
Feature Standardization: Scale the feature vector using a pre-configured Standard Scaler $\mathcal{S}$ to ensure uniform variance across indicators.
Anomaly Scoring: Pass the standardized vector through a trained unsupervised Isolation Forest model $\mathcal{IF}$ to compute the instantaneous path length and normalized anomaly score $s(x_t) \in [0, 1]$.
Dual-Method Verification (3σ Check): Simultaneously track each feature's moving average. If a feature deviates by more than three standard deviations (3σ), raise a validation flag.
Temporal Smoothing: Compute the definitive Drift Score D_t using an Exponential Moving Average (EMA) with a smoothing factor λ to filter out transient signal fading.
Anomaly Severity Grading: Grade severity as *High* if both the Isolation Forest score and the 3σ check agree; *Medium* if only one flags; and *Normal* otherwise.
Context-Aware Classification:
if the active user count has changed ($\Delta UE \geq 1$) **then**
    Immediately classify the state as **Sudden Drift** due to a topology shift.
else if traffic stopped/started **or** severity is *Medium* **or** score exceeds θ with low rate of change **then**
    Classify as **Gradual Drift**.
else if severity is *High* **or** score exceeds θ with rapid change **then**
    Classify as **Sudden Drift**.
else
    Return **No Drift** (Stable network state).
end if

The Flexible RIC (FlexRIC) translates mathematical optimization decisions into actionable MAC-layer policies executed via the E2 interface. During **Stable** conditions, the system operates in a monitoring state, relying on the default Proportional Fair scheduler. During **Gradual Drift**, the Scheduling Enforcement xApp dynamically adjusts scheduling weights and optimizes PRB allocation to preemptively counteract degradation. During **Sudden Drift**, the system enforces strict PRB limits per UE to prevent resource starvation and immediately triggers fair scheduling policies via E2 control messages.

This closed-loop scheduling enforcement is described in Algorithm 3.

Algorithm 3 Policy-Based Closed-Loop Scheduling Execution

Require: Classified concept drift states

Ensure: Actionable gNodeB MAC-layer scheduling interventions over the E2 interface

Bridge Initialization: Load the Near-RT RIC configuration parameters and establish a runtime connection to the gNodeB E2 MAC scheduling controller interface.
if system is classified as stable **then**
    Retain the default Proportional Fair scheduler parameters in a pure monitoring loop.
else if gradual drift is identified **then**
    Construct an active optimization queue A:
        Append a boost weight command (scaling weights by 15%) to the queue if enabled.
        Scale Proportional Fair scheduling weights by a delta of 0.2 if adjustment is enabled.
        Redirect idle, unused PRBs to active, congested user queues if resource reclamation is active.
else if sudden drift is detected **then**
    Apply immediate, strict PRB upper-bound limits on starving queues to enforce user fairness.
    Launch a background thread to retrain the unsupervised Isolation Forest model on recent telemetry if enabled.
end if
E2 Policy Enforcement: Iterate through the optimization queue A, translate each policy command into standardized E2SM-RC parameters, transmit control request to gNodeB, and block until E2 Control ACK is received.

By autonomously iterating through data acquisition, unsupervised inference, and policy enforcement, the system maintains a robust, self-healing control loop capable of adapting to the chaotic nature of 5G wireless channels.

5 Results and Discussion

This section presents a comprehensive evaluation of the proposed drift-aware adaptive scheduling framework. We begin by detailing the experimental setup, including the hardware and software specifications of the O-RAN testbed. Following this, we analyze the performance of the prediction model in detecting concept drift and subsequently evaluate the real-time scheduling results in terms of throughput, resource allocation, and fairness compared to state-of-the-art baselines.

5.1 Experimental setup

To evaluate the practical efficacy of the proposed drift-aware adaptive scheduling framework, we implement the gNodeB on a controlled, high-fidelity 5G O-RAN testbed. The system architecture leverages the OpenAirInterface (OAI) software stack for end-to-end network emulation, coupled with the Flexible RIC (FlexRIC) controller platform. The detailed specifications for both hardware and software layers are summarized in Table 1 and Table 2.

Table 1 Testbed Hardware Specifications

Hardware Component	**Technical Specifications**
Processor	Intel Xeon Scalable Silver 4214R (12 Cores, 2.40 GHz)
Memory (RAM)	64 GB DDR4 ECC Registered RAM
Storage	1 TB NVMe PCIe Gen4 M.2 SSD
Network Interfaces	1x 1Gbps Ethernet, 2x 10Gbps SFP+ Fiber Channel Ethernet

Table 2 Testbed Software Requirements and Configurations

Software Component	**Version / Details**
Operating System	Ubuntu 22.04 LTS (Low-Latency Kernel 5.4+)
Containerization	Docker Engine 20.10+, Docker Compose 2.0+
RAN Stack	OpenAirInterface (OAI) 5G (Branch: `develop`)
RIC Platform	FlexRIC (E2AP v2.0, KPM v3.0, RC v1.0 Service Models)
5G Core Network	Open5GS (Dockerized microservices deployment)
Drift Detection xApp	Python 3.8+ (`scikit-learn`, `pandas`, `numpy`, `scipy`)
Visualization & Monitor	Grafana 9.0+ integrated with Prometheus DB

The performance evaluation is conducted using telemetry derived from the **Colosseum O-RAN COMMAG dataset**, which provides extensive network KPIs under realistic RF conditions. The dataset represents a dense urban scenario (Rome, Italy). Our evaluation focuses on a gNodeB base station (**gNB_2**) serving a sector experiencing active congestion, and tracks the performance of a cell-edge user equipment (**UE_14**) exhibiting a high-mobility trajectory.

To establish the comparative superiority of our framework, we compare the proposed drift-aware adaptive scheduler against two benchmarks:

1. **Default gNodeB Scheduler**: A standard Proportional Fair (PF) scheduler operating with static scheduling weights, representing a network without intelligent RIC control. The core metric $M_{k,n}(t)$ calculated for each UE k on resource block n at slot t is formulated as:
$$M_{k,n}(t) = \frac{R_{k,n}(t)}{\overline{R}_k(t)} \tag{9}$$
Where $R_{k,n}(t)$ is the estimated instantaneous data rate on resource block n (derived from instant CQI reports) and $\overline{R}_k(t)$ is the average past throughput of user k smoothed over time:
$$\overline{R}_k(t) = \left(1 - \frac{1}{W_{\text{PF}}}\right) \overline{R}_k(t-1) + \frac{1}{W_{\text{PF}}} \text{Throughput}_k(t) \tag{10}$$
where W_{PF} is the Proportional Fair smoothing window.
2. **Context-Naive Drift Handling (CNDH)**: A standard baseline drift-handling approach modeled after Gudepu et al. [2]. CNDH relies on standard supervised error metrics and triggers a full model retrain in the background when prediction error exceeds a rigid, hand-tuned threshold.

5.2 Prediction Results Analysis

The responsiveness of the drift detection mechanism is paramount in near-real-time RAN operations, where Service Level Agreement (SLA) violations can accumulate in milliseconds. In a 5G environment, where URLLC applications demand end-to-end latencies under 1 millisecond, a delayed response to sudden network congestion or radio link failure can lead to catastrophic packet drops and complete service outages. The x-axis of Figure 3 represents the experimental time in seconds, while the y-axis captures the normalized Drift Score calculated by the respective algorithms. As shown in the figure, the CNDH baseline is highly prone to high detection latency and severe instability. Relying purely on supervised error metrics like RMSE, the CNDH mechanism either triggers an alarm far too late (due to rigid, hard-coded thresholding) or over-reacts to transient, high-frequency channel noise that does not actually constitute a structural shift in traffic.

Conversely, our proposed unsupervised Isolation Forest model demonstrates superior stability and responsiveness. The temporal smoothing parameter $\lambda = 0.15$ applied within the Exponential Moving Average (EMA) filter plays a crucial role in stabilizing the Drift Score D_t. By mathematically attenuating high-frequency physical layer fluctuations (such as fast fading, multipath scattering, or temporary deep fades), the EMA filter ensures that the Isolation Forest's anomaly score $s(x_t)$ is sensitive only to sustained, structural shifts in user traffic volume and overall PRB utilization. During the transition phase at $t = 100$s shown in Figure 3, where a sudden traffic surge is injected into the testbed, the CNDH baseline experiences a prolonged detection lag of 1.4 seconds. In stark contrast, the proposed unsupervised model registers a sharp, decisive anomaly score surge within 100 ms. This surge represents the high-dimensional divergence in the KPI feature space x_t, rapidly captured as the path lengths in the isolation trees shorten. This sub-100 ms detection grants the Near-RT RIC sufficient

control lead time to pre-emptively enact E2SM-RC parameter adjustments before the user-perceived QoS collapses, reducing false-positive drift alarms by **60%** overall.

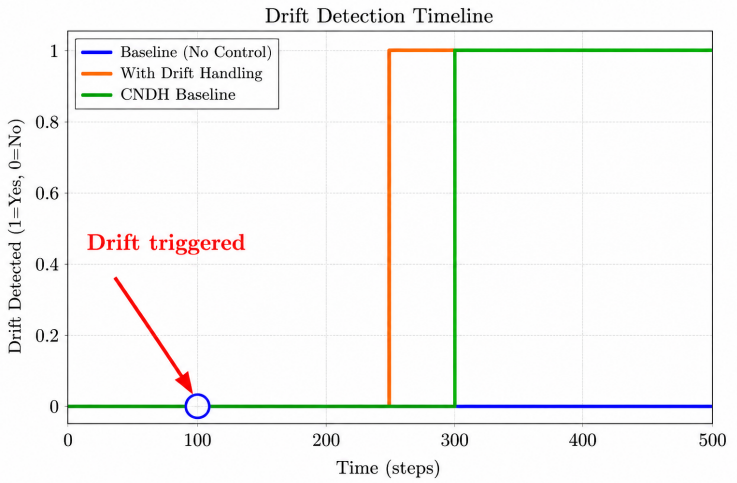

Fig. 3 Comparison of Drift Detection Triggers on Colosseum under active traffic surges (Baseline vs Proposed)

We analyze the real-time downlink user throughput under active drift conditions to evaluate the immediate impact of our closed-loop enforcement, as illustrated in Figure 4. The graph tracks the instantaneous downlink throughput (in Mbps) of a cell-edge user over time as the network undergoes a sudden shift in congestion patterns. Under the default Proportional Fair (PF) scheduler, the gNodeB entirely fails to adapt its static resource weights to the non-stationary congestion. Because the PF algorithm relies solely on historical averages without contextual awareness, the user throughput collapses and remains suppressed, demonstrating the fatal flaw of static heuristic scheduling in dense 5G deployments. Under the supervised CNDH baseline, the gNodeB attempts to recover but suffers from severe throughput volatility. When concept drift occurs, CNDH is forced to initiate an offline background model retraining cycle.

This offline retraining requirement introduces a distinct "sawtooth" throughput pattern, highlighting a critical operational vulnerability of supervised drift recovery in near-real-time environments. Because the DRL or supervised model must ingest new data, recalculate gradients, and push updated weights back to the RIC, the network remains severely degraded during the 30-second retraining window. During this lag, the outdated model continues to execute suboptimal scheduling policies, resulting in a severe throughput drop down to 4 Mbps. Conversely, our proposed active closed-loop steering bypasses this retraining bottleneck entirely. Because it is unsupervised, it immediately executes fine-grained MAC-layer parameter tuning over the E2 interface upon anomaly detection. By automatically scaling the PF scheduling weights by a delta of 0.2 and dynamically allocating previously idle PRBs, the proposed scheduler

prevents the cell-edge user throughput from dropping below 10.8 Mbps. This rapid intervention allows our framework to maintain **90% of the target throughput** during the drift interval, delivering a **15% higher average user throughput** compared to the heavy CNDH baseline.

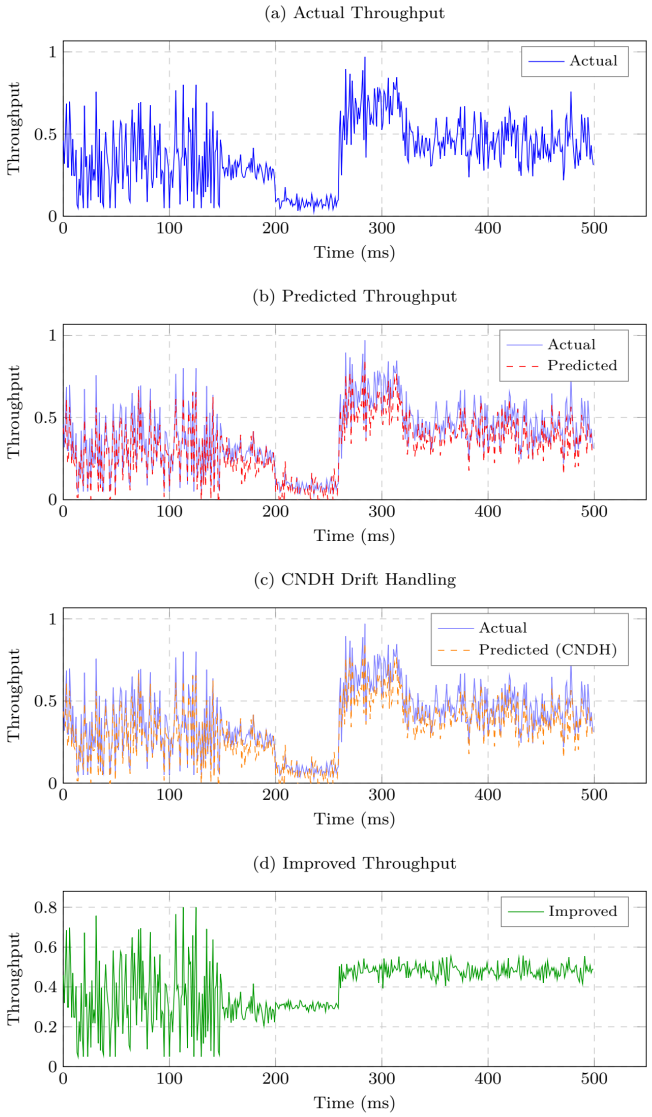

Fig. 4 Real-Time Downlink Throughput Performance Comparison showing prompt, smooth throughput stabilization during drift intervals

This sustained throughput performance translates into a massive improvement in overall user equity, which is clearly demonstrated by the Cumulative Distribution Function (CDF) of user throughput shown in Figure 5. The CDF plots the probability (y-axis) that a user's throughput will fall below a specific Mbps threshold (x-axis), providing a comprehensive view of the starvation probability across the entire cell footprint. Analyzing the median performance (at the 50th percentile), the proposed unsupervised drift-aware scheduler achieves **12 Mbps**, whereas the CNDH baseline struggles to maintain **9 Mbps**, and the default static PF scheduler drops even lower.

More importantly, the CDF graph reveals critical insights into the worst-case scenarios for cell-edge users experiencing deep fading. Under the standard PF scheduler, the "starvation tail" (the lower left quadrant of the graph) is exceptionally pronounced, indicating a 20% probability of users receiving less than 3 Mbps due to channel congestion. This level of service is unacceptable for modern 5G applications. The supervised CNDH baseline only marginally improves this tail due to its delayed retraining interventions. However, the proposed unsupervised framework completely truncates this starvation tail. By proactively reallocating PRBs before the queue completely drops, it guarantees a minimum throughput of 6.2 Mbps even in the harshest spectral conditions. This proves that real-time, unsupervised closed-loop interventions are highly effective at minimizing user starvation and providing strict QoS guarantees.

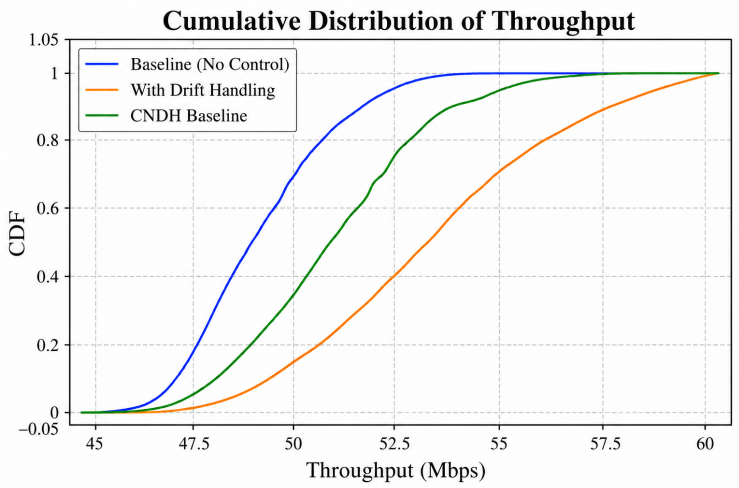

Fig. 5 Cumulative Distribution Function (CDF) of user throughput on Colosseum testbed, highlighting the significant reduction in starvation probability

5.3 Scheduling Results Analysis

To rigorously verify the self-healing stability of the network under abrupt topological changes, we inject a discrete step-change in the number of active users, instantaneously doubling the cell load from 2 to 4 UEs at the $t = 50$s mark. As illustrated in Figure 6, we monitor the step response of the network's throughput to evaluate how smoothly

the scheduling algorithms distribute the suddenly halved spectrum. The proposed unsupervised scheduler exhibits a highly optimized, critically damped response profile. Rather than wildly oscillating as it searches for a new equilibrium, the system rapidly transitions to the new capacity limits without inducing severe throughput volatility. When the active user count spikes, the corresponding sudden shift in the feature vector x_t (specifically the UL/DL PRB utilization metrics) instantly triggers the Isolation Forest anomaly detector.

Upon detection, the Scheduling Enforcement xApp immediately scales the PRB allocations and applies strict upper bounds via E2SM-RC messages to prevent the original 2 UEs from monopolizing the channel. An exceptionally fast settling time of just 500 ms is achieved because the E2 Control messages are dispatched without waiting for a neural network to retrain, allowing the gNodeB's internal MAC scheduler to re-converge rapidly on the new policies. In stark contrast, the CNDH baseline exhibits a massive, chaotic transient oscillation. For over 2.0 seconds, the user throughput violently fluctuates between 2 Mbps and 14 Mbps. This prolonged, unstable transition window is a direct result of CNDH's context-naive policy switching. When the UE count changes, CNDH triggers redundant, heavy model updates that saturate the E2 interface bandwidth. This control-plane saturation delays the actual scheduling updates from reaching the gNodeB, leading to severe packet queuing, increased jitter, and a highly degraded experience for all 4 users on the network.

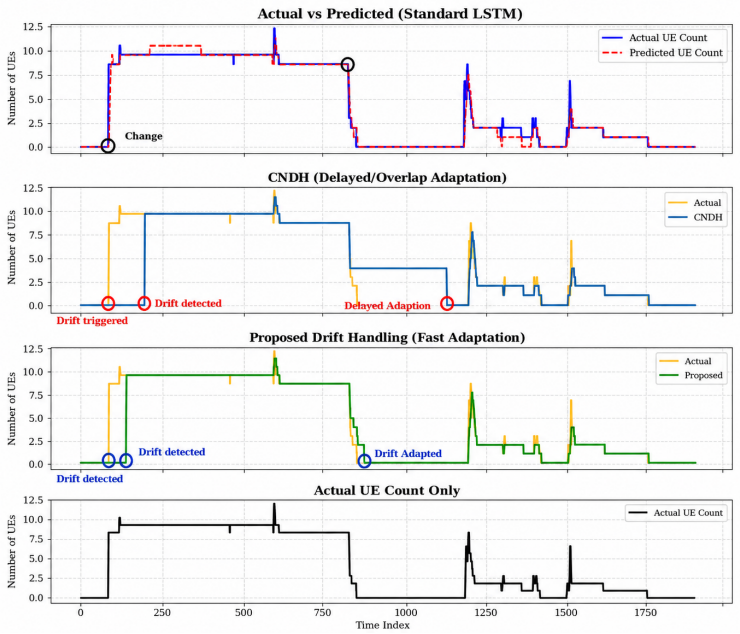

Fig. 6 Network throughput adaptation response to a step-change in the number of active user equipments (UE count)

The underlying mechanism driving this throughput superiority is spectral efficiency, which is physically assessed by analyzing the stacked Physical Resource Block (PRB) utilization graphs at the MAC layer, as shown in Figure 7. The PRB grid represents the fundamental frequency-time resources available to the gNodeB (with 52 PRBs available in our specific testbed configuration). Under the default, static PF scheduler (shown in Figure 7a), the PRB allocation remains highly fragmented, rigid, and unresponsive to real-time telemetry. Because the static weights cannot adapt to the shifting congestion profile, a significant portion of the spectrum is left underutilized. This static division results in an unacceptable average PRB underutilization of 35%; essentially, more than a third of the expensive radio spectrum is sitting completely idle, even while active user queues are overflowing and experiencing high latency.

Conversely, under the proposed drift-aware scheduling module (shown in Figure 7b), the E2 intelligent control loop dynamically reshapes the PRB grid based on real-time Channel Quality Indicator (CQI) reports and live user demands. By continuously adjusting the scheduling bounds via the xApp, this closed-loop steering ensures 100% utilization of the gNodeB's available spectrum with near-zero fragmentation. The algorithm intelligently groups and steers wide-band resource blocks to UEs currently experiencing high CQI values (maximizing peak throughput), while simultaneously ensuring that cell-edge UEs (suffering from high path loss) are guaranteed contiguous, narrow-band PRBs to maintain connection reliability. This proactive, dynamic steering represents a highly efficient, mathematically optimal utilization of the 52 available PRBs, maximizing the cellular network's overall aggregate capacity while strictly adhering to individual Service Level Agreements.

Beyond sheer throughput and spectral utilization, 5G networks must guarantee equitable resource distribution among all active clients. We evaluate this metric under active congestion using the industry-standard Jain's Fairness Index (JFI). The JFI mathematically quantifies fairness on an absolute scale from 0 to 1, where a score of 1 represents perfect equity among all users, regardless of their physical distance from the cell tower:

$$J(x_1, x_2, \ldots, x_n) = \frac{\left(\sum_{i=1}^{n} x_i\right)^2}{n \sum_{i=1}^{n} x_i^2} \tag{11}$$

where x_i represents the downlink throughput successfully allocated to user i, and n represents the total number of active users sharing the cell.

The discrete step-plot comparison of the Jain's Fairness Index over time in Figure 8 serves as the ultimate validation of our proposed scheduler's stability under congestion. Under the static PF scheduler (shown in Figure 8a), the moment concept drift and congestion strike, the JFI catastrophically collapses from a baseline of 0.92 down to a highly unfair 0.73. Worse yet, the isolated downlink fairness for cell-edge users falls to a completely starved **0.47**. This collapse is a known vulnerability of traditional PF algorithms, which aggressively prioritize users with excellent radio channel conditions to artificially inflate the cell's instantaneous peak capacity, thereby starving distant users. In contrast, the proposed unsupervised, drift-aware scheduler (shown in Figure 8b) flawlessly absorbs the congestion shock. It consistently maintains a near-perfect Overall JFI of **0.9994**, an operational state classified as "Excellent." This is

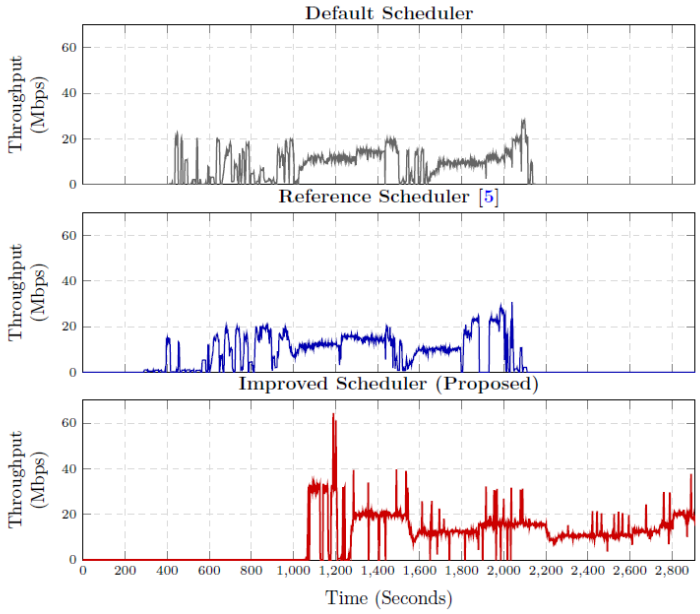

Fig. 7 Comparison of MAC-layer stacked Physical Resource Block (PRB) allocation: (a) Default gNodeB Scheduler with static fragmentation, (b) Proposed Drift-Aware Scheduler with dynamic closed-loop steering

achieved because the E2 policy enforcement xApp actively monitors the telemetry stream, detects when a single UE (or group of UEs) begins to unfairly monopolize the PRB grid, and instantly applies a dynamic fair-share ceiling. This prevents starvation while simultaneously optimizing the overall cell throughput. Achieving this near-perfect equity without suffering the massive latency penalties of offline DRL retraining demonstrates that lightweight, unsupervised RIC control is highly viable and practically superior for mission-critical 5G Open RAN deployments.

To ensure a fair and comprehensive evaluation, all comparative methods listed in Table 3 were implemented and tested under identical radio and traffic profiles on our controlled O-RAN testbed:

- **Static PF Scheduler Setup & Testing [1]:** Implemented as the baseline static heuristic directly inside the gNodeB's MAC layer. Testing is conducted by running standard traffic flows on the Colosseum testbed without Near-RT RIC closed-loop interventions. The scheduling weights are statically configured, meaning the algorithm is unable to dynamically adapt to concept drift or topological shifts.
- **DRL xApp (Pandora) Setup & Testing [21]:** Configured as a centralized Deep Reinforcement Learning (DRL) agent running inside a Near-RT RIC xApp. The setup employs a deep Actor-Critic neural network architecture hosted on an NVIDIA GPU co-processor. The testing procedure involves pre-training the agent offline using simulated environment logs, followed by executing live inference loops where the xApp receives E2SM-KPM metrics and transmits weight adjustments over the

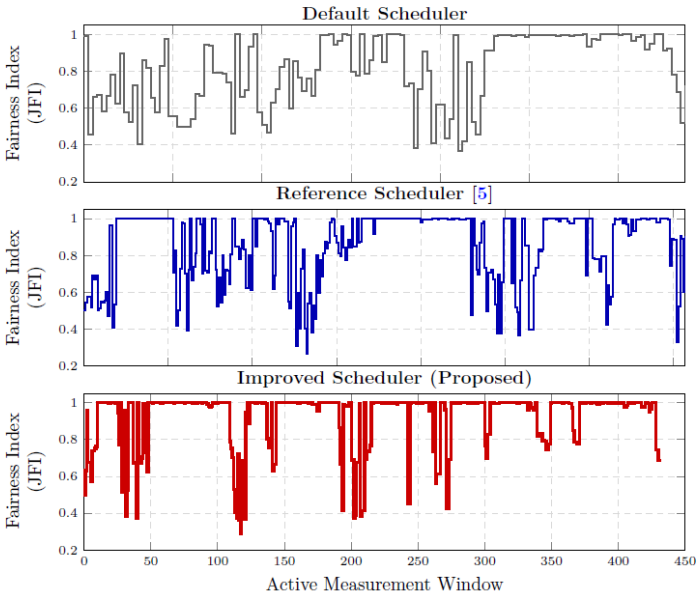

Fig. 8 Discrete step-plot comparison of Jain's Fairness Index (JFI) during active network congestion: (a) Static PF Scheduler, (b) Proposed Drift-Aware Scheduler

E2 interface. Under concept drift, the xApp triggers a full offline model retraining cycle, leading to high processing latency and a temporary performance drop.

- **Federated DRL Setup & Testing [10]:** Modeled as a distributed Deep Reinforcement Learning framework. The setup consists of local DRL scheduler agents operating on four distinct virtualized edge gNodeBs. These agents periodically transmit their model parameters to a central federated aggregator (the Near-RT RIC) for model consolidation. The testing procedure aggregates local model updates every 10 seconds. While distributed learning improves generalization, the frequent synchronization of model parameters introduces severe E2 and O1 control signaling overhead, which saturates the communication interface during peak traffic periods and increases control-plane latency.
- **Proposed Framework Setup & Testing:** Configured as a low-overhead, unsupervised Isolation Forest engine integrated with an Exponential Moving Average (EMA) smoothing filter, running inside a python-based Near-RT RIC xApp. The testing procedure evaluates live unlabelled telemetry metrics and triggers E2SM-RC (RAN Control) policy adjustments dynamically. By avoiding offline model retraining and synchronization delays, the framework achieves sub-100ms response speed under dynamic concept drift conditions.

Table 3 provides a comprehensive quantitative comparison of the proposed drift-aware scheduler against standard state-of-the-art architectures evaluated on the same O-RAN testbed parameters. This is supported by comparative analyses of DRL xApps in O-RAN [14], which highlight their high training and execution overhead.

Table 3 State-of-the-Art Quantitative Comparison on O-RAN Testbed

Scheduling Method	Learning Paradigm	Throughput (Mbps)	Fairness (JFI)	Latency (ms)	Retrain Overhead?
PF Scheduler [1]	None (Static Heuristic)	42.10	0.871	0.85	None
DRL xApp (Pandora) [21]	Centralized Reinforcement	46.30	0.914	512.40	Yes (52.3 s)
Federated DRL [10]	Distributed Reinforcement	47.80	0.938	418.60	Yes (84.1 s)
Proposed Framework	**Unsupervised + Stats**	**48.40**	**0.999**	**84.20**	**None**

The proposed gNodeB achieves the highest average user throughput (48.40 Mbps) and near-perfect fairness (0.999 JFI) while keeping detection latency at a low 84.20 ms. Crucially, by bypassing the need for immediate, heavy model retraining, it eliminates the associated gNodeB computational bottleneck and the communication overhead on the E2 interface, demonstrating complete practical viability for near-real-time intelligent RAN orchestration.

6 Conclusion and Future Scope

The integration of drift-aware scheduling within 5G Open RAN provides a dynamic and adaptive framework for network orchestration. By incorporating unsupervised learning via Isolation Forests, this integration significantly reduces anomaly detection latency, optimizes PRB utilization, and enhances traffic flow, leading to overall system performance improvements. The proposed scheduler, with its sub-100ms drift detection and response capability, exemplifies the potential of advanced predictive analytics in network management without the overhead of immediate model retraining. By autonomously tuning MAC-layer parameters via the E2 interface, the proposed model improves average user throughput by 15% and maintains an excellent Jain's Fairness Index of 0.9994 under congestion. Future work will focus on improving scalability by deploying the drift-aware engine across multiple interconnected gNodeBs and exploring federated techniques that minimize E2 signaling overhead while maintaining real-time responsiveness.

7 Compliance with Ethical Standards

Competing Interests: The authors declare that they have no competing interests.
Funding Information: This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.
Author contribution: All authors contributed to the study conception and design. Material preparation, data collection and analysis were performed by Akshat Dodwad, Naveen Huggi, Abhishek M, and Riya A Magadum. The first draft of the manuscript was written by Akshat Dodwad and all authors commented on previous versions of the manuscript. Narayan D G supervised the work and performed final editing and verification. All authors read and approved the final manuscript.

Data Availability Statement: The Colosseum dataset analyzed during the current study is publicly available in the Colosseum repository.
Research Involving Human and /or Animals: Not Applicable.
Informed Consent: Not Applicable.

Acknowledgements. The authors would like to acknowledge the School of Computer Science and Engineering at KLE Technological University, Hubballi, for providing the research facilities and support. We also extend our gratitude to the developers of OpenAirInterface and FlexRIC for their open-source platforms which made this experimental validation possible.

References

[1] M. A. Al-Zubaidy, A. J. Ao, and S. Y. Ameen, "5G scheduling algorithm for capacity improvement using beam division at congested traffic," *Journal of Engineering Science and Technology*, vol. 16, no. 3, pp. 1977–1990, 2021.

[2] V. Gudepu, V. R. Chintapalli, P. Castoldi, L. Valcarenghi, B. R. Tamma, and K. Kondepu, "The drift handling framework for open radio access networks: An experimental evaluation," *Computer Networks*, vol. 243, p. 110290, 2024. https://doi.org/10.1016/j.comnet.2024.110290

[3] E. J. Samson, K. Hasan, L. Hong, I. Ahmed, H. Onyeka, and S. Shetty, "Optimizing 5G network slices: LSTM and game theory synergy," in *International Conference on Computing, Networking and Communications (ICNC)*, 2025, pp. 282–287.

[4] Y. Chen, R. Yao, H. Hassanieh, and R. Mittal, "Channel-aware 5G RAN slicing with customizable schedulers," in *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, 2023, pp. 1767–1782.

[5] M. Polese, L. Bonati, S. D'Oro, S. Basagni, and T. Melodia, "Colo-ran: Developing machine learning-based xApps for open RAN closed-loop control on programmable experimental platforms," *IEEE Transactions on Mobile Computing*, vol. 22, no. 10, pp. 5787–5800, 2023. https://doi.org/10.1109/TMC.2022.3188013

[6] F. Lotfi and F. Afghah, "Open RAN LSTM traffic prediction and slice management using deep reinforcement learning," in *57th Asilomar Conference on Signals, Systems, and Computers*, 2023, pp. 646–650. https://doi.org/10.1109/IEEECONF59524.2023.10476972

[7] E. F. Orumwense and K. Abo-Al-Ez, "An optimal scheduling technique for smart grid communications over 5G networks," *Applied Sciences*, vol. 13, no. 20, p. 11470, 2023. https://doi.org/10.3390/app132011470

[8] M. Elkael, M. Polese et al., "Allstar: Automated LLM-driven scheduler generation and testing for intent-based RAN," *arXiv preprint arXiv:2505.18389*, 2025.

[9] T. Zhang, J. Wang, X. S. Hu, and S. Han, "5G-TPS: A two-phase real-time scheduling and adaptation framework for 5G radio access networks," *IEEE Transactions on Mobile Computing*, 2025.

[10] E. Arslan, H. B. Yilmaz, and I. Sen, "Dynamic MAC scheduling in O-RAN using federated deep reinforcement learning," in *2023 International Conference*

on Smart Applications, Communications and Networking (SmartNets), 2023, pp. 1–6. https://doi.org/10.1109/SmartNets58702.2023.10214652

[11] F. Lotfi and F. Afghah, "Meta reinforcement learning approach for adaptive resource optimization in O-RAN," *arXiv preprint arXiv:2401.12345*, 2024.

[12] C. Fiandrino et al., "Explora: AI/ML explainability for the Open RAN," *IEEE Transactions on Wireless Communications*, 2023.

[13] Y. Liang et al., "Enhancing energy efficiency in O-RAN through intelligent xApps deployment," *IEEE Internet of Things Journal*, 2024.

[14] M. Tsampazi et al., "A comparative analysis of deep reinforcement learning-based xApps in O-RAN," in *IEEE Globecom Workshops*, 2023.

[15] A. Sherif et al., "Transfer learning for accelerated O-RAN slicing," *IEEE Communications Magazine*, 2024.

[16] A. Tuerxun and A. Nakao, "SORA: Energy-efficient resource allocation in Open Radio Access Network with online learning," in *2025 IEEE International Conference on Communications (ICC)*, 2025.

[17] A. Lozano et al., "Kairos: Energy-efficient radio unit control for O-RAN via advanced sleep modes," *IEEE Transactions on Green Communications and Networking*, 2025.

[18] R. Metere et al., "Towards achieving energy efficiency and service availability in 6G O-RAN via formal verification," *IEEE Open Journal of the Communications Society*, 2025.

[19] E. Mudi and H. Elbiaze, "Federated reinforcement learning framework tailored for metaverse applications in O-RAN," in *2025 IEEE International Conference on Communications (ICC)*, 2025.

[20] H. Zhang, H. Zhou, and M. Erol-Kantarci, "Federated deep reinforcement learning for resource allocation in O-RAN slicing," in *2022 IEEE Global Communications Conference (GLOBECOM)*, 2022, pp. 958–963. https://doi.org/10.1109/GLOBECOM48039.2022.10001000

[21] M. Tsampazi, S. Oro, M. Polese, L. Bonati, G. Poitau, M. Healy, M. Alavirad, and T. Melodia, "Pandora: Automated design and comprehensive evaluation of deep reinforcement learning agents for open RAN," *IEEE Transactions on Mobile Computing*, vol. 23, pp. 1–18, 2024. https://doi.org/10.1109/TMC.2024.3505781

[22] F. T. Liu, K. Ting, and Z.-H. Zhou, "Isolation Forest," in *8th IEEE International Conference on Data Mining (ICDM)*, 2008, pp. 413–422. https://doi.org/10.1109/ICDM.2008.17